
Predicting Connectivity in Graph-Based Building Information Models Using Neural Networks

Animesh Mittal, Anurag Jain, Archi Patel, Dipesh Patel, Gareth War,
Ikjyot Singh, Maulik Srivastava, Sagar Murgan, Vagdevi Ponugupati

Department of Computing Science

University of Alberta

{animesh1, anurag5, archi1, dipesh1, war, ikjyot,
maulik2, murgan, ponugupa}@ualberta.ca

Abstract

Building Information Modeling (BIM) is essential for construction planning, but the current process of manually determining component connectivity is time-consuming and error-prone. Our work explores graph-based approaches to automate this process by predicting probabilistic connections between BIM components' nodes using Graph Neural Networks. We generate connection graphs where the edges represent the likelihood of connection, with edge values indicating the connection probability. Our models are trained on real construction data that incorporate real-world constraints such as spatial distance and component orientation. They achieve high F2 scores and outperform traditional distance-based heuristics, improving scalability and reducing manual effort in BIM workflows.

1 Introduction

BIM is widely used in construction to represent the layout and properties of the building components. It helps engineers and designers plan large projects more effectively. However, one part of this process, which is figuring out how different components are connected, is still done manually. This task is slow, can be inconsistent, and becomes harder as the models become larger and more complex.

Knowing how components such as steel beams and steel columns are connected is important for many tasks, such as checking for clashes or planning how materials (such as cables) should be routed through a building. Traditional methods rely on fixed rules or distance thresholds to decide whether two components are connected. These rules do not always work well, especially when components are of different types or positioned in less obvious ways.

In this project, we explore machine learning approaches to solve this problem. By turning the BIM into a graph and using Graph Neural Networks (GNNs), we can learn to predict whether two components are connected. Instead of using fixed rules, our method learns patterns from real construction data. This allows it to make better predictions and handle different kinds of building layouts more easily.

2 Background and Related Work

2.1 Traditional Machine Learning in BIM

Traditional machine learning (ML) techniques have been applied to BIM tasks such as clash detection and component classification. An example is the use of the Modified Extreme Gradient Boosting (MXGBoost) algorithm for clash detection by using informative features such as "material properties"

and "load specifications" [Shehadeh et al., 2024]. This approach achieved a high Area Under the Curve (AUC) score of 0.972, highlighting the potential of using ML for tasks in this field.

However, while effective, these models rely on fixed input dimensions and struggle with complex relationships between components (e.g., surroundings of a component). They also work better only for structured decision trees or tabular data, and not too well on a graph structure [Yan and Xu, 2024]. Our dataset is a graph, where each building component has a start and end node, and edges of two types: component edges (connects the start and end node of a singular component) and connection edges (connects two components). Traditional ML models struggle with graphical data, resulting in a loss of important spatial and relational information, leading to poor connectivity predictions in our BIM graphs.

2.2 Graph-Based Learning in BIM

BIM data captures complex spatial and relational information between components, which can be effectively represented as a graph. This makes Graph Neural Networks (GNNs) well-suited for modeling and predicting structural connections. GNNs can be tweaked to process both local and global relationships (e.g., GraphSAGE), making them a suitable choice for predicting structural connections [Hamilton et al., 2018].

Several papers inspired our approach for the task. Jayasinghe and Brilakis [2023] proposed methods based on GNN that achieved F1 scores above 90% when predicting connections between BIM components, outperforming rule-based methods. Wang [2023] showed that combining geometric features (e.g., bounding boxes and centroids) with semantic data (e.g., material properties) leads to better relationship inference in GNN models since they use spatial and contextual information. Austern et al. [2024] found that BIM graph representations, when enriched with contextual information significantly improved classification performance compared to traditional ML models using both relational and geometric data.

These works show that GNNs can learn meaningful connection patterns directly from the structure and semantic features of the BIM data. Unlike traditional rule-based methods, GNNs do not require hard-coded thresholds. They adapt to different types and layouts, allowing for more flexible and accurate connectivity prediction.

3 Our Approach

The input to our models is a graph with just BIM components, while the output is a graph with connection edges between said components. The input contains information on the component nodes with attributes such as Type, ComponentID, and spatial coordinates (X, Y, Z), and edges with attributes such as Component Type (Beam or Column) and Edge_ID (a unique identifier). The output produced by the models is also a graph and builds on the same attributes as the input while also drawing connection predictions between component nodes. Our approach is to experiment with individual and hybrid models to assess the suitability of GNNs for the task.

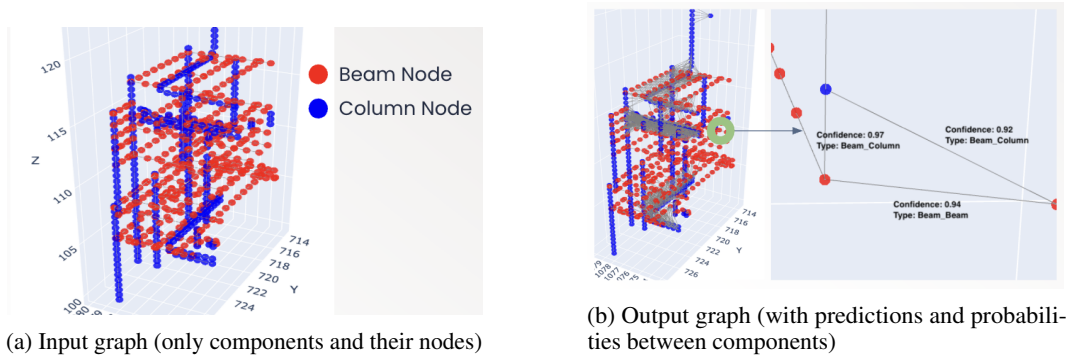


Figure 1: Comparison between input and output graph structures

3.1 Pre-Processing

We split the data into 70% and 30% subsets for training and testing, respectively. One approach randomly assigns components into both subsets. However, this approach does not preserve the building’s structure, and so was discarded. The second approach we considered was the x-coordinate split that partitions the graph at the 70% mark along the x-axis. We adopt this strategy because it preserves structural information. Three negative edges per node are added to the training set so that the model can also learn non-connection patterns.

3.2 Candidate Edge Generation

We begin by generating candidate edges between nodes belonging to different components. These edges are then passed to the models, which predict the probability of a valid connection between each pair. To generate these candidate edges, we use two approaches. The first is a K -nearest neighbors strategy, where each node is attached to nodes from other components using candidate edges, based on type-specific distance thresholds. A limit on the maximum number of neighbors (K) is applied to control the number of candidate edges per node, balancing computational efficiency and coverage. In the second approach, each node in the graph is randomly attached to a diverse set of nodes from other components through candidate edges. This ensures broad connectivity and increases the likelihood of capturing valid connections that more restrictive methods might miss.

3.3 Models

We investigate multiple different model architectures to suit our needs.

3.3.1 Graph Convolutional Network (GCN)

In this approach, a GCN equally aggregates the features of all neighbours in the graph, predicting connections based on fixed neighbourhood information [Kipf and Welling, 2016].

We extracted a set of input features for nodes and edges to capture the spatial and topological properties of the structural elements. The node features include type, clustering coefficient, and coordinates. Edge features consist of the Euclidean distance, Manhattan distance, z-difference, and its angle with respect to the x-axis. Together, these features enable the model to learn the underlying patterns of connectivity in the BIM graph.

This model consists of four GCNConv layers with hidden dimension 256, each followed by a ReLU activation. These layers aggregate node features from the graph neighbourhood, producing node embeddings that capture both local and global structural information:

$$h_v^{(l+1)} = \text{ReLU}(\text{GCNConv}(h_v^{(l)}, A))$$

where $h_v^{(l)}$ is the feature vector of node v at layer l , and A is the adjacency matrix [Kipf and Welling, 2016].

Edge connectivity is predicted using a multi-layer perceptron (MLP) that processes a 520-dimensional combined feature vector, formed by concatenating node embeddings (from the final GCN layer) with edge features. The MLP, equipped with fully connected layers, ReLU activations, and a dropout rate of 0.3 to prevent overfitting, reduces the feature dimension progressively from 520 to 256, then to 128, before outputting 3 logits corresponding to the connectivity classes (no connection, Beam_Beam, Beam_Column). A softmax activation transforms these logits into a probability distribution to predict edge connection types.

The GCN approach aggregates features from all neighbours equally, providing a simple way to make connectivity predictions. The primary reason for using GCN is the nature of our dataset, which consists of only two types of components (beams or columns) with relatively uniform connectivity patterns defined by spatial proximity and geometric alignment. Given this homogeneity and the limited diversity in node types and relationships, we opted to treat all neighbours equally rather than applying weighted contributions.

3.3.2 Graph Attention Networks (GAT)

The GAT model employs a self-attention mechanism, assigning dynamic edge-wise attention weights to prioritize more informative neighbors during message passing. This enhances the model’s ability to learn complex structural patterns in graph data and improves link prediction performance [Zhang et al., 2024].

We paired the GAT model with Euclidean distance, Coordinate Difference, Hadamard product embeddings, and Adamic-Adar scores as edge features. Let u and v denote source and target nodes, with embeddings h_u and h_v . The Hadamard product $e_{uv}^{\text{Had}} = h_u \circ h_v$ captures geometric interactions between node embeddings. The Adamic-Adar score $e_{uv}^{\text{AA}} = \sum_{w \in \mathcal{N}(u) \cap \mathcal{N}(v)} \frac{1}{\log d(w)}$ emphasizes structural similarity by prioritizing rare shared neighbors w . Here, $d(w)$ is the degree of w , $\mathcal{N}(u)$ and $\mathcal{N}(v)$ denote the neighborhoods of nodes u and v , respectively [De et al., 2013].

To model this data, we stacked four GAT layers. The first three apply multi-head attention (four heads k), while the fourth outputs unified embeddings. Let x denote node features and e denote edge attributes. The GAT updates embeddings through the operation

$$x_k = \text{GATConv}_k(\text{ELU}(x_{k-1}), \text{edge_index}), \quad k = 1, 2, 3, 4, \quad x_0 = x \quad (1)$$

where each layer computes attention coefficients as

$$\alpha_{uv} = \text{softmax}_v \left(\text{LeakyReLU} \left(a^\top [Wh_u \parallel Wh_v] \right) \right) \quad (2)$$

with α_{uv} denoting the learned importance of node v ’s features for node u , using shared weight matrix W and a learnable vector a . Here, the Wh_u and Wh_v are the linearly transformed embeddings of the source and target nodes.

The final edge representation is obtained by concatenating the node embeddings from the last GAT layer with edge features and passing them through an MLP. This outputs logits, which are converted to probabilities via softmax. An edge is predicted if its probability exceeds 0.9.

3.3.3 GraphSAGE

GraphSAGE [Hamilton et al., 2018] is an inductive GNN that generates node embeddings by sampling a local neighbourhood of a fixed size and aggregating the features of the neighbours. It learns a parametric function that can embed previously unseen nodes and entirely new graphs without retraining. This makes it well-suited for BIM applications, where new components are frequently added and the graph structure evolves over time.

Each node v is represented by a standardized feature vector $\mathbf{x}_v$, encoding spatial coordinates and material properties. We uniformly sample a fixed-size subset at each layer to bound computation and memory. Edge attributes (e.g., Euclidean distance) are removed from traditional GraphSAGE but incorporated later in the link-prediction head.

At each layer of the model k , the embedding of node v is updated according to

$$h_v^{(k)} = \sigma \left(W^{(k)} \left[h_v^{(k-1)} \parallel \text{AGG}_{u \in \mathcal{N}_S(v)}^{(k)} (h_u^{(k-1)}) \right] \right).$$

Where $h_v^{(0)} = x_v$ is the raw feature vector, σ is a non-linear activation function (ReLU), $W^{(k)}$ are the trainable weights, AGG is the mean aggregator and $\mathcal{N}(v)$ is a sampled subset of neighbours. After K layers, the final node embedding is $\mathbf{z}_v \equiv \mathbf{h}_v^{(K)}, \forall v \in V$ (set of all nodes), thus encoding information from nodes up to K -hop away. We employ the mean aggregator to preserve feature scale and reduce overfitting in BIM graphs.

To predict whether an edge (u, v) should exist, we concatenate the node embeddings

$$\mathbf{s}_{uv} = [\mathbf{z}_u \parallel \mathbf{z}_v],$$

combining all edge attributes with the help of the link prediction head. The resulting vector passes through a linear layer followed by a sigmoid activation. We classify (u, v) as connected if $p_{uv} > 0.95$.

GraphSAGE’s inductive nature allows on-the-fly embedding of new BIM elements without retraining. By capturing multi-hop structural context and using a mean aggregator, it balances between local and global properties while incorporating edge attributes using the link predictor.

3.3.4 Message Passing Neural Networks (MPNN)

We design a customized MPNN to predict clashes in BIM graphs by leveraging spatial features and structural relationships between the building components. The input nodes include normalized 3D coordinates (X, Y, Z) and a categorical component type (e.g., beam, column) in a 4D vector. Edges, labeled connections, or non-connections, capture spatial or structural relationships.

The MPNN employs three custom message-passing layers with hidden dimensions of 64, 32, and 16, following the message-passing neural network framework [Gilmer et al., 2017], augmented with dropout (0.01) to mitigate overfitting. Each layer performs:

- **Message:** Computes messages via a learnable linear transformation:

$$m_{u \rightarrow v} = \text{ReLU}(W_m x_u),$$

where x_u denotes the original features of neighbor node u (e.g., a 4D vector of normalized 3D coordinates and component type) and W_m maps input features to hidden dimensions.

- **Aggregation:** Sums neighbour messages:

$$a_v = \sum_{u \in \mathcal{N}(v)} m_{u \rightarrow v}.$$

- **Update:** Concatenates the x_v (Node’s original features) with a_v :

$$h_v = \text{ReLU}(W_u [x_v; a_v]),$$

where W_u transforms the concatenated vector.

Unlike GCNs, which aggregate self and neighbour features into a normalized sum early, our update preserves x_v , enabling distinct weighting of component-specific features (e.g., a beam’s Z-coordinate) against neighbourhood context. A two-layer MLP link predictor concatenates node embeddings for edge pairs, transforming them to output clash probabilities.

The MPNN’s custom update rule, which preserves node-specific features until the final step, outperforms GCNs for sparse BIM graphs by balancing individual coordinates and neighbourhood context, enhancing rare clash detection.

3.3.5 Hybrid GCN

This approach integrates a GCN backbone with an encoder-decoder framework. The input node features form a 4D vector with normalized spatial coordinates (X, Y, Z) and a numerically encoded component type (e.g., Beam or Column) via `LabelEncoder`, while edge features, the Euclidean distance, explicitly influence connectivity predictions through spatial proximity.

Encoder: Two `GCNConv` layers aggregate hierarchical neighbour information. The update rule is: [Kipf and Welling, 2016]:

$$\mathbf{h}_v^{(l)} = \sigma \left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{|\mathcal{N}(v)| |\mathcal{N}(u)|}} \mathbf{h}_u^{(l-1)} \mathbf{W}^{(l)} \right),$$

where $\mathbf{h}_v^{(l)}$ is the embedding at layer l , $\mathbf{h}_u^{(l-1)}$ is the embedding at layer $l - 1$ ($\mathbf{h}_v^{(0)} = \mathbf{x}_v$, the initial features), $\mathcal{N}(v)$ is the neighbour set, $\mathbf{W}^{(l)}$ is a learnable weight matrix, and σ denotes a ReLU activation. The normalization factor $\frac{1}{\sqrt{|\mathcal{N}(v)| |\mathcal{N}(u)|}}$ accounts for node degrees to prevent scaling issues. The first layer maps 4D input to 1280D latent space with ReLU, and the second produces a refined 640D representation, encapsulating multi-hop structural patterns.

Decoder: Predicts edge connectivity using node embeddings created by the encoder and edge distances:

1. Distance is transformed to e_{uv} via $(\text{Linear}(1, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, 640))$.
2. The similarity between node embeddings z_u and z_v is quantified as $s_{uv} = z_u \odot z_v$.
3. Concatenated $[s_{uv}, e_{uv}]$ feeds into an MLP, the output logit is converted to probability p_{uv} via sigmoid activation. Edges are classified as connected if $p_{uv} > 0.9$.

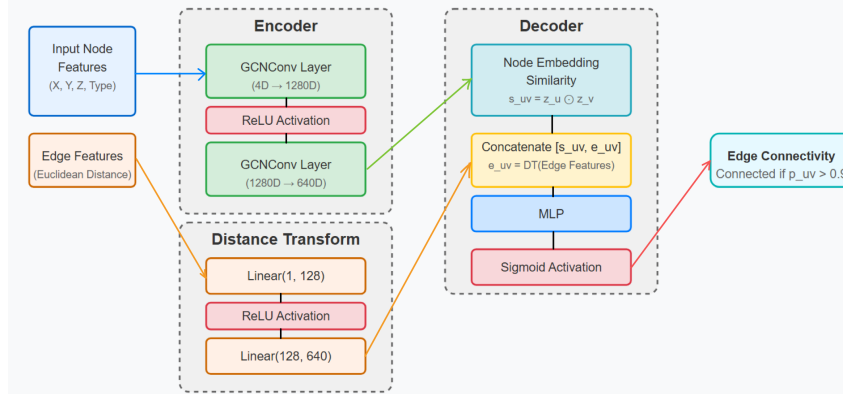


Figure 2: Illustration of the Hybrid GCN model architecture.

The Hybrid GCN was chosen for the encoder’s ability to capture hierarchal patterns, and the decoder’s ability to see structural dependencies through node embedding similarity scores.

3.3.6 Hybrid GCN-GAT Model

This hybrid model integrates GCNs and GATs, achieving high accuracy in predicting connections between components in BIM graphs. The input node features are identical to those in the **Hybrid GCN** (Section 3.5.1), forming a 4D vector of normalized spatial coordinates (X, Y, Z) and a numerically encoded component type. Unlike Hybrid GCN, edge features are not used.

Encoder: The encoder uses one GCNConv layer [Kipf and Welling, 2016], aggregating local neighbour information to create initial embeddings. The embedding for each node v is computed as in the Hybrid GCN (Section 3.5.1), where $h_u^{(0)} = x_u$ is the initial feature vector of node u , and σ denotes a ReLU activation. A **GATv2 layer** [Veličković et al., 2018] refines these embeddings by adaptively assigning attention coefficients to neighbours, producing 64-dimensional embeddings that capture complex relational patterns.

Decoder: Connectivity predictions use the refined embeddings. The node embedding similarity score $s_{uv} = z_u \odot z_v$ is computed as in Hybrid GCN (Section 3.5.2), followed by sigmoid activation to obtain probability p_{uv} . Edges are classified as connected if $p_{uv} > 0.9$.

By combining GCN’s efficient local aggregation with GAT’s dynamic attention, the hybrid model delivers precise component connectivity predictions.

4 Evaluation

4.1 Data

The dataset from PCL Construction consists of a spreadsheet and a ground truth graph. The spreadsheet contains columns detailing information about the components. The connections in the ground truth were manually annotated by engineers.

Further analysis of the spreadsheet and ground truth revealed a significant limitation: the spreadsheet lacked sub-nodes (which were generated by PCL during the ground truth creation process). Sub-nodes account for scenarios where the end of one component intersects the midpoint of another (e.g., a beam clashing with the center of a column). Directly using the spreadsheet for prediction leads to evaluation discrepancies. This is because any generated sub-nodes will have mismatched IDs in comparison to the ground truth, resulting in false positives and missed true positives. To tackle this, we reverse-engineered the ground truth to create inputs with no connections for our models.

4.2 Experimental Setup

We train each model using the Adam optimizer (learning rate: 0.001) and a class-weighted CrossEntropyLoss to address the imbalance caused by the abundance of nodes that are not connected. By assigning higher weight to "no connection" class, the model is better at reducing false positives.

4.3 Predicted Output

The output of each model is a graph that represents the connectivity between structural elements within the BIM environment. For each test edge created (as described in Section 3.2), the model predicts a probability indicating the likelihood of a valid connection. If this probability exceeds a specified threshold (e.g., 0.9), the edge is classified as a positive connection and labeled as either Beam_Beam or Beam_Column, based on the types of the connected nodes. This graph also contains multiple connections between two components.

4.4 Evaluation Consideration

Since connections in the ground truth are manually annotated by engineers (as mentioned in Section 3), they often select only one representative edge among multiple potential connections between two components. This results in an under-representation of true connection edges. As multiple connections would facilitate downstream applications (such as path-finding algorithms), PCL prefers our model predicts multiple connections.

Consequently, to recover those omitted edges, we adopt a recall-focused evaluation metric such as the F2 score.

$$F2 = \frac{(1 + 2^2) \text{Precision} \times \text{Recall}}{2^2 \text{Precision} + \text{Recall}}$$

Although this approach may reduce precision, it increases the likelihood of capturing important connections.

4.5 Evaluation Setup

We evaluated all models on a real-world BIM dataset provided by PCL Construction. As mentioned in Section 3.1, we used an x-axis spatial split into a 70% training and 30% testing subsets. The output graph from each model is evaluated against a ground truth graph using metrics such as Precision, Recall, F1 score, and F2 score. These scores were then compared with a rule-based baseline model that determined connections based on a specified threshold distance.

4.6 Model Comparison and Discussion

Model	Precision	Recall	F1 Score	F2 Score
Baseline	0.3248	0.9500	0.4841	0.6859
GAT	0.6222	0.8182	0.7069	0.7698
GraphSAGE	0.6674	1.0000	0.8005	0.9094
MPNN	0.7164	0.9350	0.8200	0.8835
GCN	0.5293	1.0000	0.6922	0.8490
Hybrid GCN	0.7603	0.9578	0.8477	0.9104
Hybrid GCN + GAT	0.7205	0.9253	0.8102	0.8755

Table 1: Comparison of models on the test set

According to Table 1, GNN-based approaches significantly outperformed the rule-based baseline model, confirming their ability to learn complex connectivity patterns. The Hybrid GCN achieved the highest F2 score by capturing hierarchical spatial context with its encoder and combining learned embedding similarity with explicit Euclidean distance in its decoder. GraphSAGE also performed well, using mean aggregation over sampled neighbourhoods to achieve perfect recall, though its lower precision indicates some over-prediction. The MPNN demonstrated strong performance, benefiting from its unique update rule that preserves original node features alongside aggregated messages, enabling nuanced decisions. The Hybrid GCN+GAT was also effective, though the GATv2 layer's

dynamic attention provided less benefit than the Hybrid GCN’s explicit distance handling for this specific geometric task. The standard GAT showed moderate results, suggesting attention alone was less suited compared to other mechanisms. Finally, while the standard GCN achieved perfect recall, leveraging the dataset’s relative homogeneity, its uniform neighbour aggregation resulted in low precision. This indicates it struggled to distinguish finer spatial relationships and generated more false positives.

5 Conclusion and Discussion

Our experiments demonstrate that GNNs offer significant improvements in predicting component connectivity in BIM environments. Compared to rule-based model, where connection is established based on a distance threshold, GNNs can generalize across spatial configurations and handle complex structures without relying on hard-coded thresholds.

The GNN models evaluated in Section 4.6 effectively leverage spatial and structural relationships for clash detection, enhancing construction planning automation.

Future research could explore promising directions, including extending predictions to non-steel components (e.g., trays, conduits), evaluating 3D spatial interaction risks (e.g., identifying blockages like “**Close**: that fire suppression pipe blocks the door, 0.85” or violations like “**Violation**: that stairwell has no lighting, 0.70”), and integrating richer features, such as bounding boxes, to improve clash detection accuracy.

By advancing automated connection prediction, this work paves the way for more efficient, scalable, and error-resistant construction planning, transforming digital workflows in the architecture, engineering, and construction industry.

Acknowledgments

We sincerely thank Professor Russ Greiner for his invaluable guidance and mentorship throughout the course of this project. We are also grateful to our teaching assistant, Hobie Lee, for his consistent support and feedback. Special thanks to our domain experts—Brian Gue, Orion Sehn, and Christoph Sydora—for their insightful inputs that significantly shaped our understanding. Finally, we extend our appreciation to all team members for their dedication, collaboration, and commitment to the successful completion of this project.

References

- Guy Austern, Tanya Bloch, and Yael Abulafia. Incorporating context into bim-derived data—leveraging graph neural networks for building element classification. *Buildings*, 14(2), 2024. ISSN 2075-5309. doi: 10.3390/buildings14020527.
- Abir De, Niloy Ganguly, and Soumen Chakrabarti. Discriminative link prediction using local links, node features and community structure. In *2013 IEEE 13th International Conference on Data Mining (ICDM)*, pages 1009–1018, 2013. doi: 10.1109/ICDM.2013.68.
- Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272. PMLR, 2017.
- William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs, 2018. URL <https://arxiv.org/abs/1706.02216>.
- Haritha Jayasinghe and Ioannis Brilakis. Learnable geometry and connectivity modelling of bim objects. In *34th British Machine Vision Conference 2023, BMVC 2023, Aberdeen, UK, November 20-24, 2023*. BMVA, 2023. URL <https://papers.bmvc2023.org/0305.pdf>.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *CoRR*, abs/1609.02907, 2016. URL <http://arxiv.org/abs/1609.02907>.

- Ali Shehadeh, Odey Alshboul, Madhar M. Taamneh, Aiman Q. Jaradat, and Ahmad H. Alomari. Enhanced clash detection in building information modeling: Leveraging modified extreme gradient boosting for predictive analytics. *Results in Engineering*, vol. 24, 2024. doi: 10.1016/j.rineng.2024.103439.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. URL <https://arxiv.org/abs/1710.10903>.
- Zijian Wang. Graph representation of bim models. In *Proceedings of the 2023 European Conference on Computing in Construction and the 40th International CIB W78 Conference*, volume 4 of *Computing in Construction*, Heraklion, Greece, July 2023. European Council on Computing in Construction. ISBN 978-0-701702-73-1. doi: 10.35490/EC3.2023.333.
- Liuxi Yan and Yaoqun Xu. Xgboost-enhanced graph neural networks: A new architecture for heterogeneous tabular data. *Applied Sciences*, 14(13), 2024. ISSN 2076-3417. doi: 10.3390/app14135826. URL <https://www.mdpi.com/2076-3417/14/13/5826>.
- Zhiwei Zhang, Xiaoyin Wu, Guangliang Zhu, Wenbo Qin, and Nannan Liang. A graph attention network-based link prediction method using link value estimation. *IEEE Access*, 12:34–45, 2024. doi: 10.1109/ACCESS.2023.3346688.